College Timetable Generator Architecture and Design Specification

Project: College Timetable Generator

Version: 1.0

Authors: Harshitha B. T, Diya B R, Gaurav S, Harsha K R

Date: 27 October 2026

Status: Draft for academic review

Revision History

Version | Date | Author | Change Summary
1.0 | 27-10-2026 | Diya B R, Harshitha B. T, Gaurav S, Harsha K R | Project-specific architecture and design draft

Approvals

Role | Name | Signature/Date
Project guide | Pending | Pending

1. Introduction

1.1 Purpose
This document describes the architecture, interfaces, constraints, and user experience of the College Timetable Generator. It is based on the Flask application, scheduler, tests, and CI workflow in the project repository.

1.2 Scope
The application accepts course offerings and weekly scheduling settings through a web form or CSV upload, generates a timetable, and presents it as a weekly grid by class section. It includes teacher availability and one fixed classroom per section. It does not store schedules in a database, authenticate users, or deploy the application.

1.3 Audience
The primary users are timetable coordinators who enter offerings and settings, and students or staff who read the generated schedule. Developers, testers, and the project instructor can use this specification to understand and review the implementation.

1.4 Definitions
Course code: the university identifier shown to users. Subject offering: a course taught by a

teacher to one class section. subject_id: an internal unique identifier for an offering. Slot: a one-based period within a configured day. Teacher availability: the allowed day-slot pairs for a listed teacher. Hard constraint: a rule that must never be violated, such as no teacher, class, or room clash.

2. Document Overview

2.1 How to use this document

Sections 3 and 4 describe the implemented architecture and request flows. The diagrams are text renderings of the components and sequences. Appendix A maps key terms and repository artifacts. Statements about behavior reflect the current source; items under Open Issues are not implemented guarantees.

2.2 Related Documents

The repository artifacts are README.md, SRS, SAD, Test Plan.

3. Architecture

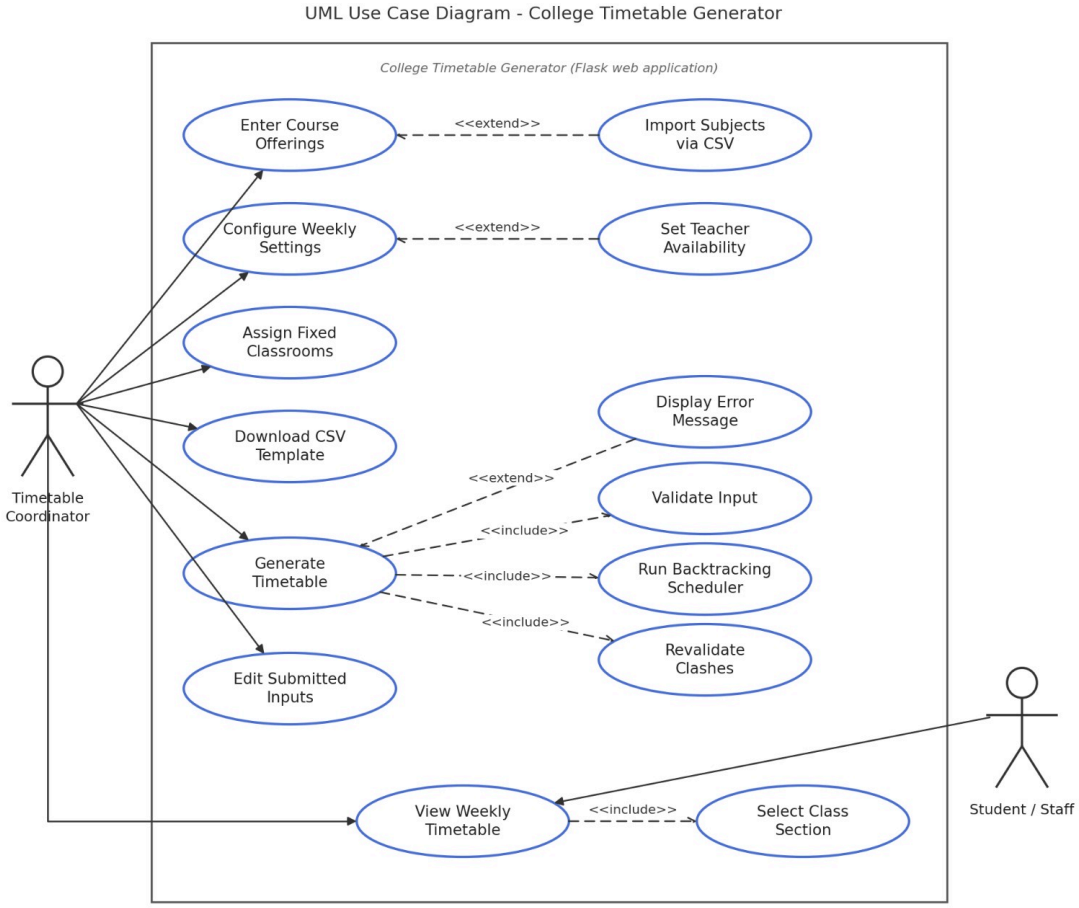
3.1 Goals & Constraints

Goals: schedule each offering's requested weekly periods when feasible; prevent simultaneous teacher, class, and room conflicts; respect allowed teacher slots; preserve one assigned room per section; and show results in a responsive weekly grid. Constraints: the scheduler accepts subjects, a room list, ordered days, slots_per_day, and optional teacher availability. Slots are one-based. No complete schedule raises ValueError. The Flask layer maps fixed rooms after scheduling and runs the independent timetable validator again.

3.2 Stakeholders & Concerns

Timetable coordinator: clear input, validation, and fixed classroom assignment.
Students and staff: readable day-by-slot results for the selected section.
Teachers: no overlapping assignments and respect for configured allowed slots.
Developers and testers: a small modular codebase, clear scheduler contract, and repeatable automated tests.
Project instructor: traceable design and evidence that the repository's workflow runs tests.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Browser UI: templates/index.html and templates/results.html render the input form and section-filtered weekly timetable. static/style.css provides responsive and theme styling; static/app.js handles row controls, theme, section selection, and form feedback.

Flask application: app.py parses manual rows or CSV, validates settings, maps rows into scheduler subjects, calls generate_timetable, applies fixed rooms, validates the result, and builds the display grid.

Scheduler: app/scheduler.py expands periods_per_week into scheduling tasks and uses backtracking with hard conflict checks. It returns session dictionaries.

Constraint layer: app/constraints.py checks teacher, class, and room use in the same day and slot, validates session fields, and independently checks the completed list.

CI workflow: .github/workflows/ci.yml installs requirements on Ubuntu with Python 3.12 and runs the full pytest suite. It does not deploy the application.

3.5 Chosen Architecture Pattern and Rationale

The project uses a modular monolith with a browser presentation layer, Flask request and mapping layer, scheduling logic, and constraint validation. This keeps the mini-project easy to run and test while separating the core scheduling rules from HTML presentation. The Flask route still coordinates several steps directly, and the UI depends on the scheduler's returned dictionary fields.

3.6 Technology Stack & Data Stores

Runtime: Python and Flask (requirements specify Flask >=3.1,<4.0). UI: Jinja templates, HTML, CSS, and browser JavaScript. Scheduling: app/scheduler.py and app/constraints.py. Tests: pytest (requirements specify >=8.0,<10.0). CI: GitHub Actions on ubuntu-latest with Python 3.12; it installs requirements.txt and runs python -m pytest -v for pushes, pull requests targeting main, and manual dispatch. No database or automated deployment target is configured. The Flask debug server is for local development only.

3.7 Risks & Mitigations

Risk: backtracking search can become slow as offerings and constraints grow. Mitigation: the scheduler orders tasks by teacher availability and prunes conflicting placements; measure larger input cases before production use. Risk: candidate slots are visited in configured day and slot order, with no explicit balance optimization, so early weekdays may be favored. Risk: Flask replaces scheduler-selected rooms with fixed section rooms after generation; the second validator is essential to reject clashes. Risk: no authentication, CSRF protection, upload-size limit, persistence, or production deployment configuration is present.

3.8 Traceability to Requirements

Offering completeness -> app/scheduler.py expands each periods_per_week value and the Flask result mapping joins sessions back to course names and codes.
Teacher/class/room clash prevention -> app/constraints.py has_conflict and validate_timetable, used during search and after Flask fixed-room mapping.
Teacher availability -> app.py parses allowed teacher, day, slot lines; app/scheduler.py restricts candidate slots.
Fixed room by class section -> app.py resolves one room per class, maps it onto every session, and revalidates the timetable.
Weekly grid by section -> app.py builds class/day/slot data; templates/results.html renders it.
Repeatable verification -> tests/test_scheduler.py and tests/test_flask_app.py run in .github/workflows/ci.yml.

3.9 Security Architecture

The application handles user input and an optional CSV in memory and does not configure a database or authentication. Jinja templates escape values by default. The code does not show CSRF protection, a request-size limit for CSV uploads, or production security settings.

Do not expose the debug development server publicly. Before public deployment, define authentication and authorization, CSRF and upload limits, HTTPS and hosting configuration, and an operational data-retention policy. Avoid entering sensitive student or staff information unless an approved deployment protects it.

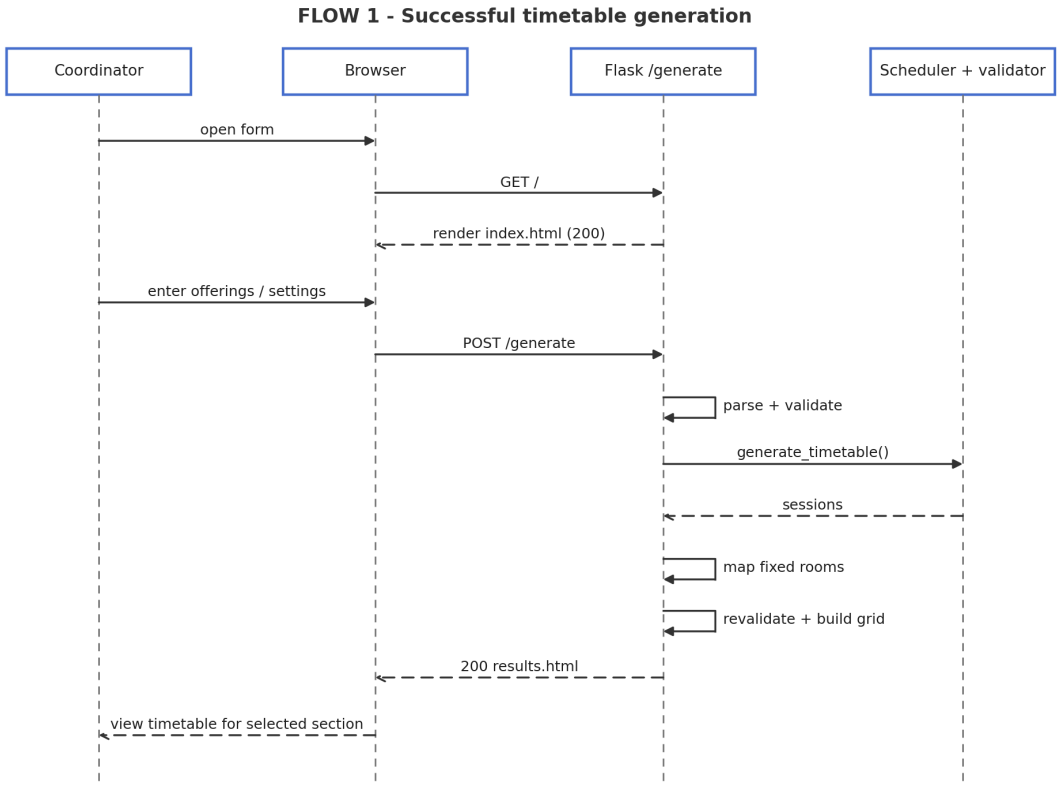
4. Design

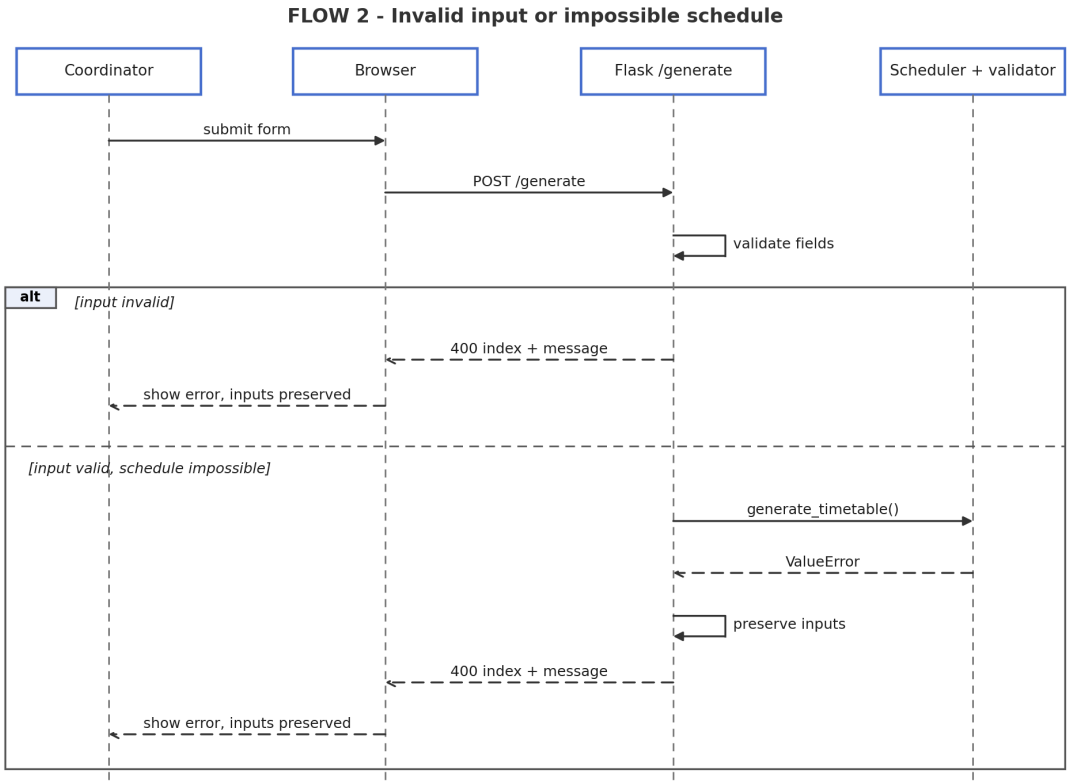
4.1 Design Overview

The browser submits configuration to Flask. Flask validates it, normalizes each offering, calls the scheduler, applies the selected fixed classroom per section, and independently rechecks clashes. It then maps course names and codes onto session rows, builds a grid for every configured day and slot, and renders results.html. Schedule data is returned in the response and is not saved to a database.

4.2 UML Sequence Diagrams

The following two sequences show a successful generation and a validation or scheduling failure. They use the actual Flask routes and scheduler boundary.





4.3 API Design

The public browser interface is HTML form and page navigation. The scheduling function is an internal Python interface, not a network API.

GET / -> renders the input form (200). GET /subjects-template.csv -> downloads a CSV header template. POST /edit -> returns to the form with submitted settings restored. POST /generate -> accepts days, slots_per_day, rooms, repeated course_code, subject_name, teacher, class_id, and periods_per_week values; optional subjects_csv upload; classroom_mode and classroom_assignments; color_mode and subject_color; and optional availability_enabled plus teacher_availability lines. A successful request renders a weekly grid with 200. Invalid input or an impossible schedule is shown on the form with 400. Internal scheduler: generate_timetable(subjects, rooms, days, slots_per_day, teacher_availability=None). Each subject has a unique subject_id, teacher, class_id, and positive periods_per_week. It returns a list of dictionaries containing day, slot, subject_id, teacher, class_id, and room. The UI maps internal IDs to the official course code and subject name for display.

4.4 Error Handling, Logging & Monitoring

Expected validation and scheduling failures are raised as ValueError, caught by the Flask generate route, and displayed as a user-facing message with HTTP 400 while retaining

submitted settings. The scheduler performs conflict checks as it searches; Flask validates again after assigning fixed rooms. Unexpected exceptions are not converted into a domain error and may produce a server error. The repository configures automated tests in CI but no runtime logging, alerting, or production monitoring service.

4.5 UX Design

The entry page presents a compact course-offering table, manual entry or CSV import, working days, slot capacity, room assignments, and optional teacher availability. The results page provides a class-section selector, an immediately visible classroom label, and a horizontally scrollable weekly grid with days as columns and slots as rows. The theme uses warm neutral and green tones with a dark theme option, visible focus styling, and small-screen layout rules. Unscheduled cells are shown as free periods.

4.6 Open Issues & Next Steps

Before production use, define where and how the Flask app will be hosted and whether deployment automation is required; the repository currently provides CI testing but no CD job. Test scheduler performance and weekday distribution on representative college data. Add authentication and CSRF protection, a CSV size limit, production error handling, and deployment-specific configuration if the app will be exposed beyond a trusted local environment.

5. Appendices

5.1 Glossary: course code, subject offering, class section, slot, teacher availability, hard constraint, CI, CD.
5.2 References: IEEE 42010, OWASP, NIST SP 800-160.
5.3 Tools: draw.io